

Group 61

Name	Roll No.	Percentage Involvement
Sanchit Mathur	2023DC04337	100
Shahid Hussain M	2023DC04389	100
Soumya Ranjan Sahu	2023DC04257	100
Ravuri Sai Vidya Sagar	2023DC04087	100

Problem Formulation

Problem Statement:

Given a dataset of customer attributes and behaviors, the objective is to develop a predictive model that accurately identifies customers at high risk of churn. Furthermore, the goal is to leverage the insights gained from this model to formulate actionable strategies for mitigating churn and improving customer retention.

Specific Objectives:

1. Develop a Predictive Model:

- Construct a machine learning model capable of accurately predicting whether a customer will churn based on their historical data.
- Evaluate the model's performance using appropriate metrics (e.g., accuracy, precision, recall, F1-score).
- Optimize the model to maximize its predictive accuracy and generalizability.

2. Identify Key Churn Drivers:

- Analyze the dataset to determine the key factors that contribute to customer churn.
- Quantify the impact of each factor on the likelihood of churn.
- Provide insights into the underlying reasons for customer attrition.

3. Formulate Churn Mitigation Strategies:

- Develop actionable strategies based on the model's predictions and the identified churn drivers.
- Propose targeted interventions to retain at-risk customers.
- Recommend improvements to customer service, product offerings, or pricing strategies to reduce churn.

4. Improve Customer Retention:

- Implement the formulated churn mitigation strategies.
- Monitor the impact of these strategies on customer retention rates.

- Continuously refine the strategies based on performance data and feedback.

Evaluation Metrics

Model Performance Metrics

- **Accuracy:** Percentage of correctly predicted churn and non-churn cases.
- **Precision:** Proportion of correctly predicted churn cases out of all predicted churn cases.
- **Recall (Sensitivity):** Proportion of correctly predicted churn cases out of all actual churn cases.
- **F1-Score:** Harmonic mean of precision and recall.
- **ROC-AUC:** Area under the ROC curve to evaluate the model's ability to distinguish between churn and non-churn classes.

Key Considerations:

- **Data Quality:** Ensure the accuracy, completeness, and consistency of the customer data.
- **Feature Engineering:** Create relevant features that capture customer behavior and preferences.
- **Model Selection:** Choose a machine learning model that is appropriate for the data and the prediction task.
- **Interpretability:** Prioritize model interpretability to facilitate the identification of churn drivers.
- **Actionability:** Focus on developing strategies that can be practically implemented by the business.

Expected Outcomes:

- A trained machine learning model capable of predicting customer churn.
- A report detailing the key churn drivers and their impact.
- A set of actionable strategies for mitigating churn and improving customer retention.
- Improved customer retention rates and increased customer lifetime value.

Data Ingestion

Raw Data Storage

Imports:

- `google.cloud.storage`: Imports the Google Cloud Storage client library, which allows Python to interact with GCS.
- `datetime`: Imports the datetime module for working with dates and times, used for partitioning the data.
- `os`: Imports the os module for interacting with the operating system, used for file path manipulation and setting environment variables.
- `from prefect import flow`: imports prefect flow to create a workflow.
- `from src.logger import logging`: Imports a custom logging module for logging messages.

Setting Google Application Credentials:

- `key = r'D:\ML-Projects\Customer-Churn\notebook\data\key.json'` : Defines the path to the Google Cloud service account key file.
- `os.environ["GOOGLE_APPLICATION_CREDENTIALS"] = key`: Sets the `GOOGLE_APPLICATION_CREDENTIALS` environment variable to the path of the key file. This allows the Google Cloud Storage client to authenticate. **Important:** Never hardcode your key file path or put your key file in version control.

upload_to_gcs Function:

- **Arguments:**
 - `bucket_name`: The name of the GCS bucket.
 - `source_file_path`: The local path of the file to upload.
 - `destination_folder`: The base folder in the GCS bucket where the file will be uploaded.
- **Functionality:**
 - `client = storage.Client()`: Initializes a GCS client using the credentials set in the environment variable.
 - `bucket = client.bucket(bucket_name)`: Gets a reference to the specified GCS bucket.
 - `current_date = datetime.utcnow().strftime('%Y/%m/%d')`: Gets the current UTC date in the format YYYY/MM/DD, used for partitioning the data.
 - `file_name = os.path.basename(source_file_path)`: Extracts the file name from the source file path.
 - `destination_blob_path = f'{destination_folder}/{current_date}/{file_name}'`: Constructs the destination path in GCS, including the base folder, date partition, and file name.
 - `blob = bucket.blob(destination_blob_path)`: Creates a blob (object) in the GCS bucket with the specified destination path.

- `blob.upload_from_filename(source_file_path)`: Uploads the local file to the created blob.
- `logging.info(...)`: Logs a message indicating that the file was successfully uploaded.

if `__name__ == "__main__"`: Block:

- This block executes when the script is run directly.
- `BUCKET_NAME`, `SOURCE_FILE_PATH`, and `DESTINATION_FOLDER` are defined with example values. **You must replace these with your actual values.**
- `upload_to_gcs(BUCKET_NAME, SOURCE_FILE_PATH, DESTINATION_FOLDER)`: Calls the `upload_to_gcs` function to upload the file.

Data Validation

Imports:

- `dataclasses`: For creating data classes.
- `numpy as np`: For numerical operations.
- `pandas as pd`: For data manipulation using DataFrames.
- `src.components.data_ingestion.DataIngestion`: Imports the `DataIngestion` class from a custom module.
- `reportlab`: For generating PDF reports.
- `src.logger.logging`: Imports the custom logging module.
- `src.exception.CustomException`: Imports a custom exception class.
- `prefect import flow`: imports prefect flow to create a workflow.

`generate_data_quality_report(df)` Function:

- Takes a Pandas DataFrame `df` as input.
- Creates a report dictionary to store data quality metrics.
- **Missing Values Check**: Calculates the percentage of missing values for each column.
- **Data Type Validation**: Checks if the actual data types of columns match the expected data types.
- **Outlier Detection**:
 - Identifies numeric columns.
 - Calculates the first quartile (Q1), third quartile (Q3), and interquartile range (IQR).

- Detects outliers using the $1.5 * IQR$ rule.
- Stores the count of outliers for each numeric column.
- **Duplicate Records Check:** Counts the number of duplicate rows.
- **Logging:** Logs the generated report.
- Returns the report dictionary.

`export_data_quality_report_to_pdf(report, file_name="data_quality_report.pdf")` Function:

- Takes the report dictionary and an optional `file_name` as input.
- Creates a PDF document using `reportlab`.
- Adds a title to the report.
- Iterates through the report dictionary and adds each section to the PDF.
- Handles dictionary data (e.g., incorrect data types, outliers) by creating tables.
- Formats the tables with styles.
- Builds the PDF document.
- Logs a message indicating successful export.

`if __name__ == "__main__":` Block:

- Creates an instance of the `DataIngestion` class.
- Calls the `call()` method of the `DataIngestion` instance to ingest data.
- Checks if the data ingestion was successful.
- If successful, it logs sample data and the shape of the `DataFrame`.
- Calls the `generate_data_quality_report()` function to create a data quality report.
- Calls the `export_data_quality_report_to_pdf()` function to export the report to a PDF file.

Data Preparation

Imports:

- `dataclasses`: For creating data classes.
- `numpy as np`: For numerical operations.
- `pandas as pd`: For data manipulation using `DataFrames`.
- `src.components.data_ingestion.DataIngestion`: Imports the `DataIngestion` class.
- `src.logger.logging`: Imports the custom logging module.
- `src.exception.CustomException`: Imports a custom exception class.

- matplotlib.pyplot as plt: For creating plots.
- seaborn as sns: For statistical data visualization.
- os: For operating system interactions.
- sys: For system-specific parameters and functions.
- sklearn.impute.SimpleImputer: For handling missing values.
- sklearn.preprocessing.LabelEncoder, StandardScaler: For encoding categorical variables and scaling numerical features.
- prefect import flow: imports prefect flow to create a workflow.

DataPreparationConfig Dataclass:

- Defines a data class to hold the configuration for data preparation, specifically the path to the raw data file.

DataPreparation Class:

- **__init__(self):** Initializes the class with a DataPreparationConfig instance.
- **prepare_data(self, df: pd.DataFrame) -> pd.DataFrame:**
 - Takes a Pandas DataFrame as input.
 - Converts float columns to integers where applicable.
 - Converts the "churn" column to string labels ('False', 'True').
 - Identifies numerical and categorical columns.
 - Handles missing values:
 - Uses SimpleImputer with the "most_frequent" strategy for categorical columns.
 - Uses SimpleImputer with the "median" strategy for numerical columns.
 - Removes duplicate rows.
 - Encoding categorical features with LabelEncoder.
 - Scaling numerical features with StandardScaler.
 - Generates visualizations:
 - Histograms of numerical feature distributions.
 - Box plots for outlier detection.
 - Count plot of the "churn" distribution.
 - Returns the cleaned and prepared DataFrame.
- **call_prep(self) -> pd.DataFrame:**
 - Handles data ingestion and preparation in sequence.
 - Creates an instance of DataIngestion.
 - Calls the call() method of DataIngestion to fetch the raw data.
 - If the data ingestion is successful, it calls prepare_data() to clean and prepare the data.
 - Returns the processed DataFrame or None if ingestion fails.

if __name__ == "__main__": Block:

- Creates an instance of DataPreparation.
- Calls the call_prep() method to perform data ingestion and preparation.
- Logs the results, including the first few rows and the shape of the cleaned data.
- Saves the generated plots to the img folder.

Data Transformation and Storage

Imports:

- dataclasses: For creating data classes.
- numpy as np: For numerical operations.
- pandas as pd: For data manipulation using DataFrames.
- src.components.data_preparation.DataPreparation: Imports the DataPreparation class.
- reportlab: For generating PDF reports (though not used in this version).
- os: For operating system interactions.
- google.cloud.secretmanager: For accessing secrets from Google Cloud Secret Manager.
- json: For working with JSON data.
- snowflake.connector: For connecting to Snowflake databases.
- prefect import flow: imports prefect flow to create a workflow.
- src.logger.logging: Imports the custom logging module.

DataTransformationConfig Dataclass:

- Defines a data class to hold the configuration for data transformation, specifically the path to the raw data file.

DataTransformation Class:

- **__init__(self):** Initializes the class with a DataTransformationConfig instance.
- **transformedData(self, df):**
 - Takes a Pandas DataFrame as input.
 - Performs feature engineering:
 - Calculates "Avg Spend Per Tenure."
 - Calculates a "Recency Score."
 - Calculates a "Churn Risk Score."
 - Calculates a "CLV" (Customer Lifetime Value) approximation.

- Saves the transformed DataFrame to a CSV file.
- Returns the transformed DataFrame.
- **datastore(self, df):**
 - Takes a Pandas DataFrame as input.
 - Retrieves Snowflake connection credentials from Google Cloud Secret Manager.
 - Connects to Snowflake using the retrieved credentials.
 - Creates a table named CustomerChurnData in Snowflake if it doesn't already exist.
 - Inserts the data from the DataFrame into the CustomerChurnData table.
 - Commits the changes and closes the connection.
- **call(self):**
 - Creates an instance of DataPreparation.
 - Calls call_prep() to get the cleaned data.
 - Creates an instance of DataTransformation.
 - Calls transformedData() to transform the data.
 - If the transformation is successful, it logs the results and calls datastore() to load the data into Snowflake.
 - Handles the case where data ingestion fails.

if __name__ == "__main__": Block:

- Creates instances of DataPreparation and DataTransformation.
- Calls call_prep() and transformedData() to process the data.
- Calls the call() method of the DataTransformation object.

Feature Store

Imports:

- snowflake.connector: Imports the Snowflake connector for connecting to Snowflake.
- pandas as pd: Imports Pandas for data manipulation.
- from prefect import flow: imports prefect flow to create a workflow.
- from src.logger import logging: Imports a custom logging module.

get_customer_features(customer_id) Function:

- Takes a `customer_id` as input.
- **Connects to Snowflake:**
 - Establishes a connection to Snowflake using provided credentials.
 - It is very bad practice to hardcode credentials. In a production environment, you should use environment variables or a secret manager.
- **Constructs the SQL Query:**
 - Creates an SQL query to select all columns from the `CUSTOMERCHURNTRANSFORMED` table where `CustomerID` matches the provided `customer_id`.
- **Executes the Query and Fetches Data:**
 - Uses `pd.read_sql()` to execute the query and load the result into a Pandas `DataFrame`.
- **Closes the Connection:**
 - Closes the Snowflake connection.
- **Returns Data:**
 - Converts the `DataFrame` to a list of dictionaries using `df.to_dict(orient='records')`. This format is convenient for JSON serialization and other data exchange.
- **Error Handling:**
 - A `try...except` block is added to catch potential exceptions during the Snowflake connection or query execution.
 - If an exception occurs, it logs an error message and returns `None`.

if `__name__ == "__main__"`: Block:

- This block is executed when the script is run directly.
- **Calls the Function:**
 - Calls `get_customer_features()` with the `customer_id "1"`.
- **Logs and Prints the Result:**
 - Logs the returned `customer_features` using the custom logging module.
 - Prints the `customer_features` to the console.

Data Versioning

Using GitHub for data versioning, while not its primary purpose, can be done with some considerations and limitations. Here's a breakdown of how we have approach it,

How to Use GitHub for Data Versioning:

- **Small Data Files:**
 - GitHub is best suited for small, text-based data files (e.g., CSV, JSON).
 - Store these files directly in your repository.
 - Commit changes with descriptive messages to track modifications.
- **Large Data Files (Limitations):**
 - GitHub has limitations on file sizes (100MB per file, 1GB per repository).
 - For larger files, consider these options:
 - **Git LFS (Large File Storage):**
 - Git LFS is designed for versioning large files.
 - It stores large files outside the repository and replaces them with pointers.
 - This keeps your repository size manageable.
 - To use Git LFS you must first install it, then you can track specific file extentions.
 - **External Storage:**
 - Store large data files in external storage (e.g., cloud storage like AWS S3, Google Cloud Storage, Azure Blob Storage).
 - Keep metadata or pointers to these files in your GitHub repository.
 - This is the most common practice.
- **Data Organization:**
 - Organize your data files in a logical directory structure.
 - Use meaningful file names and extensions.
 - Create a README.md file to document the data and its structure.
- **Data Pipelines:**
 - If your data is generated by a pipeline, automate the process of committing changes to GitHub.
 - Use tools like GitHub Actions to trigger data updates and commits.
- **Data Diffs:**

- GitHub's diffing capabilities are useful for tracking changes in text-based data files.
- This allows you to see what has been added, removed, or modified.

Model Building

Imports:

- `os`: For operating system interactions.
- `pandas` as `pd`: For data manipulation.
- `dataclasses`: For creating data classes.
- `src.logger.logging`: Imports the custom logging module.
- `src.exception.CustomException`: Imports a custom exception class.
- `sklearn.model_selection.train_test_split`: For splitting data into training and testing sets.
- `sklearn.linear_model.LogisticRegression`: For logistic regression modeling.
- `sklearn.ensemble.RandomForestClassifier`: For random forest modeling.
- `xgboost.XGBClassifier`: For XGBoost modeling.
- `sklearn.metrics`: For model evaluation metrics.
- `src.components.data_transformation.DataTransformation`: Imports the `DataTransformation` class.
- `src.components.data_preparation.DataPreparation`: Imports the `DataPreparation` class.
- `joblib`: For saving and loading models.
- `prefect import flow`: imports prefect flow to create a workflow.
- `src.logger.logging`: Imports the custom logging module.

modelBuildingConfig Dataclass:

- Defines a data class to hold the configuration for model building.

modelBuilding Class:

- `__init__(self)`: Initializes the class with a `modelBuildingConfig` instance.
- `evaluate_model(model, X_train, X_test, y_train, y_test)` (Static Method):
 - Takes a model, training data, and testing data as input.
 - Trains the model on the training data.
 - Makes predictions on the testing data.
 - Calculates and returns accuracy, precision, recall, and F1 score.

if __name__ == "__main__": Block:

- **Data Preparation and Transformation:**
 - Creates instances of `DataPreparation` and `DataTransformation`.
 - Calls `call_prep()` and `transformedData()` to prepare and transform the data.
- **Data Splitting:**
 - Converts the "churn" column to numerical values (0 and 1).
 - Splits the data into training and testing sets.
- **Model Training and Evaluation:**
 - Defines a dictionary of models to train.
 - Iterates through the models, calls `evaluate_model()`, and stores the results in a dictionary.
 - Creates a Pandas DataFrame from the results and prints it.
- **Model Saving:**
 - Saves the last model that was trained to a file named "customer_churn_model.pkl" using `joblib.dump()`.
 - **Important:** The code currently saves the last model from the loop, not necessarily the best performing model. You should add logic to select the best model based on your chosen metric (e.g., F1 score, accuracy).

Orchestrating the Data Pipeline

Imports:

- `prefect`: For defining and running Prefect flows and tasks.
- `subprocess`: For running external Python scripts.
- `src.logger.logging`: Imports the custom logging module.

Task Definitions:

- Each component of the data pipeline (data ingestion, data storage, etc.) is defined as a Prefect task using the `@task` decorator.
- `retries=3, retry_delay_seconds=5`: Configures each task to retry up to 3 times with a 5-second delay between retries.
- Each task uses `subprocess.run()` to execute the corresponding Python script.
- `capture_output=True, text=True, check=True`: Configures `subprocess.run()` to capture the script's output, treat it as text, and raise an exception if the script returns a non-zero exit code.

- Error handling: Each task includes a try...except block to catch subprocess.CalledProcessError exceptions, log errors, and re-raise the exception to signal task failure to Prefect.
- Each task returns the flow name, and most return the stdout of the subprocess.

Flow Definition:

- churn_prediction_pipeline(): Defines the main Prefect flow using the @flow decorator.
- The flow calls each task in sequence, passing the output of the previous task to the next.
- logging.info(df): Logs the final output of the pipeline.

if __name__ == "__main__": Block:

- Executes the churn_prediction_pipeline() flow when the script is run directly.